可判定性先于维护：什么真正决定了 LLM 智能体是否用到正确的记忆

李瀚
成都工业学院，成都 610000，中国 · 2499371088@qq.com

摘　要：大语言模型智能体越来越多地在会话之间保留长期记忆，但当记忆中同时存在互相矛盾的记录时，究竟是什么决定了智能体能否用对，目前并不清楚。本文构造了一个受控任务：记忆库中同一槽位同时存在过期值与更新值，要求智能体报告当前值。在 37 个条件、约 51 000 次查询、4 个模型配置上，维护策略（最近优先、重要度优先、衰减复习）所能解释的差异，远小于本文用“可判定性”概括的三个呈现层因素。记录带有可靠时间戳时，过期误用率几乎为零（最高 0.2%）；去掉时间戳后，仅记录顺序就足以让错误率在 0% 与 100% 之间翻转，且翻转方向因模型而异：DeepSeek 系列取末条，GLM-4-Flash 取首条，GLM-4-Plus 没有稳定规律。表述一致性进一步调节该效应，且值在句中的位置比来源标签或句式更重要。复习策略的收益恰好等于它清除的冲突比例：误用率等于 1 减去归档覆盖率。真实日志复现了同样的模式；据此实现的“可判定性感知记忆层”在端到端多轮任务上达到 100% 完全正确，而追加式日志为 0%。本文认为记忆系统应把可判定性作为一等属性，记忆评测必须报告时间戳、记录顺序与表述一致性。

关键词：大语言模型智能体；长期记忆；记忆维护；位置偏置；评测

1　引言

会记忆的智能体，是一个能在会话之间保存事实的智能体[9,10]：用户的服务端口、项目目录、工作流所绑定的模型。这类存储并不是只增不改的。事实会被新的取值取代，而旧记录往往仍然留着——删除有信息损失，存储又足够便宜，保留就成了默认做法。当智能体之后要回答问题时，它必须先判断哪一条记录是当前的。

一个自然的假设是：把维护做得更好就能解决这个问题，比如按时间远近、按重要度、或按遗忘曲线给记录排序，再把“正确的那条”送到模型面前。相当多的工作沿着这一假设展开，从按衰减加权的检索，到分页式记忆层级，再到生产环境中的记忆组件。本文直接检验这个假设，发现它能解释的部分少得惊人。

我们构造了一个刻意最小化的任务：记忆库对同一槽位保存两条记录，一条过期、一条更新，模型只需回答当前值。任务只有单跳、上下文很短、只问一个问题——如果维护策略真的重要，这里最应该看得出来。在 37 个条件、约 51 000 次查询、4 个模型配置上，我们看到的恰恰相反：决定答案的不是记忆被如何维护，而是这两条记录在呈现上是否“可判定”。

起主导作用的是三个呈现层因素。可靠的时间戳几乎消灭了过期误用（最高 0.2%）。把时间戳拿掉之后，仅凭记录顺序就能让错误率在 0% 与 100% 之间翻转；往哪个方向翻取决于模型，DeepSeek 系列取末条，GLM-4-Flash 取首条，GLM-4-Plus 两者都不是。表述一致性接着决定这条位置规则被执行得有多严格，而值在句中的位置又比来源标签或句式更重要。相比之下，维护策略的收益恰好等于它清除了多少冲突记录，仅此而已：在四种策略上，观测到的误用率都等于 1 减去归档覆盖率。

这些结果重新定义了工程上的问题。与其问“哪种复习策略最好”，记忆系统更应先问“这条记录到底能不能被裁决”，并据此设计存储。本文最后给出一份简短的设计准则，以及本应更早发现这些效应的评测做法。

本文的贡献如下。我们构造了受控任务与 37 个条件的设计，把维护策略与呈现方式分离开来；我们证明时间戳起决定作用，而在缺少时间戳时，位置决定答案，方向则因模型而异；我们分解了表述效应，发现值在句中的位置比来源标签或句式更重要；我们证明维护的收益等于其清除冲突的比例，并在真实日志语料上复现了该模式；最后，我们提出可判定性感知记忆层，并在端到端任务上做了检验。

2 相关工作

智能体记忆。MemGPT[3] 把上下文窗口当作主存，在外部存储之间换入换出。Generative Agents[1] 用时间远近、重要度与相关性给记忆打分，其中时间项是指数衰减。MemoryBank[2] 用艾宾浩斯式[12]遗忘曲线刻画记忆强度。生产环境的记忆组件[8]则加入抽取、合并与图结构。这些系统在存储与检索上各不相同，但共享一个本文要检验的假设：当记录互相冲突时，更好的排序或更好的复习会带来更好的答案。

长期记忆评测。LoCoMo[5] 与 LongMemEval[6] 在很长的对话历史上考察回忆与推理，衡量的是智能体能否找到并使用信息，而每个事实通常只有一条标准答案。本文任务的差别正好落在“过期”这件事上：记忆库里刻意放了一条相互矛盾的记录，问题在于模型相信哪一条。

位置效应。语言模型对相关信息出现在何处敏感，这一点已有共识，包括信息位于长上下文中间时性能明显下降[4]。本文把这一结论推进到冲突记忆：位置不只是一个让准确率浮动几分的检索先验；在没有时间戳时，它可以完全决定答案，而且方向随模型而变。

维护与间隔重复。SM-2、FSRS、半衰期回归[7]等调度器决定学习者何时复习某一项目。本文借用这一框架，但把它用在智能体上，并给出关于“调度”本身的否定结论：复习带来的收益可以归因于冲突记录的清除，而不是复习动作，也不是挑选项目所用的衰减信号。

3 任务与实验设计

3.1 任务

每个记忆库包含 N 条事实链。一条链由同一（实体，属性）槽位的两条记录构成：一条在 t_old 写入的过期记录，一条在 t_new 写入的更新记录，且 t_old < t_new；此外还有占用预算的干扰记录。查询时刻固定在第 200 天，更新发生在查询前 d 天，d 取 {1, 7, 30, 90}。干扰实体从不被提问。

对每条链，模型收到该实体检索到的记录，以短模板（“[2026-05-28] service-00·port = 8000”）或自然句呈现，并回答一个问题：当前取值是什么？答案等于更新值记为正确，等于过期值记为过期误用，其余记为未知。主指标是过期误用率。

3.2 维护策略

在相同的复习预算（记忆字符数的 40%）下比较四种策略：（1）不复习；（2）最近优先；（3）重要度优先；（4）衰减优先，即挑选留存率 $w = \exp(-\Delta t / H)$ 最低的记录，半衰期 H 按重要度分档取 {30, 12, 6} 天，记录被再次提及时有 +0.35 的回弹。复习一条记录，意味着把它与更新的记录对照：若已被取代则归档，否则刷新其时间戳。复习之后的检索遵循固定规则，因此不同策略之间唯一的差别是“复习了哪些记录”。

3.3 实验因素

时间表示：绝对日期、相对记法（“第 N 天”）、或完全没有时间戳。上下文顺序：新记录在前、旧记录在前、或随机。表述一致性：全部记录使用同一模板、自然句但句式统一、或自然句但句式混杂；另有三组单因素变体，分别只改变来源标签、只改变句式、只改变值在句中的位置。模型：deepseek-chat、deepseek-reasoner、glm-4-flash、glm-4-plus，温度均为 0，输出很短（推理模型上限 800 token）。

3.4 真实日志语料

为排除“合成字符串造成的假象”，我们从第一作者智能体的会话日志中挖掘一个确实随时间变化的配置值：某桌面智能体界面的本地端口，共观测到 25 次、14 个不同取值。相邻观测构成 13 条更新链。记忆记录就是原始日志行，逐字保留；只有提问与打分是我们的。

3.5 统计

每个条件使用 40 条链、4 个延迟、3 个随机种子（真实语料为 13 条链、3 个种子）。报告各延迟上的均值，以及跨种子的 95% 置信区间（t 分布）。整套实验在一台工作站上完成，API 花费仅数美元。由于任务确定性强、对照差异大，我们直接报告效应量，而不是依赖显著性检验。

4 结果

4.1 时间戳让问题消失

表 1 给出模板语料上的主结果。带有绝对日期时，模型几乎不会错，且没有任何维护策略能在此基础上更进一步：不复习为 0.2%，三种策略均为 0.0%。这个天花板并不是题目变简单了，而是日期本来的作用。相对记法已经弱得多：不复习时平均 15.8%，在 90 天延迟上升到 50.8%，说明模型并不能可靠地把“第 110 天”与“第 65 天”映射到先后关系上。

表 1　模板语料上的过期误用率（%），deepseek-chat，40 条链 × 4 个延迟 × 3 个种子。越低越好。

条件	不复习	最近优先	重要度优先	衰减优先
绝对日期，新在前	0.2	0.0	0.0	0.0
绝对日期，旧在前	0.0	0.0	0.0	0.0
相对记法（第 N 天）	15.8	4.8	4.2	4.8
无时间戳，新在前	100.0	100.0	55.4	47.5
无时间戳，旧在前	0.0	0.0	0.0	0.0

4.2 没有时间戳时，位置决定答案，且方向因模型而异

去掉时间戳、保持表述完全一致，结果非常干脆：更新记录在前时误用率为 100%，更新记录在后时为 0%。每个不复习条件下的 480 次查询都符合位置规则；把顺序打乱后，误用率落在两者之间（34.8%，在混杂表述下测得），与“顺序是起作用线索”的判断一致。模型并不是在推理哪条记录更新，它读的是位置。

表 2 表明这条规则的方向是模型的属性，而不是任务的属性。两个 DeepSeek 模型都取末条。GLM-4-Flash 取首条：更新在前时误用率 0%，更新在后时 100%。GLM-4-Plus 又是另一种情形，分别为 49.4% 与 30.4%，即高于随机但没有可识别的稳定规则。加上日期后，四个配置全部回到 0%，从而把失败定位在“缺少可判定的线索”上。

表 2　无时间戳时的过期误用率（%），按模型与记录顺序，均为不复习基线。末行为 13 条真实配置变更链。

模型 / 语料	更新在前	更新在后	带日期
deepseek-chat	100.0	0.0	0.2
deepseek-reasoner	100.0	—	0.0
glm-4-flash	0.0	100.0	0.0
glm-4-plus	49.4	30.4	0.0
deepseek-chat，真实日志	89.1	8.3	0.0

4.3　表述影响显著，其中值的位置最重要

正是表述一致才让位置规则具有确定性，因此我们进一步问：究竟哪一方面的表述会破坏它。从统一基线出发（deepseek-chat 为 100%，glm-4-flash 为 0%），每次只改变一个维度（表 3）。对 deepseek-chat 而言，只改来源标签降到 90.4%，只改句式降到 85.6%，只改值在句中的位置降到 70.2%；三者同时混杂时为 65.0%。这些效应大致可加（9.6 + 14.4 + 29.8 ≈ 53 个百分点），同时改变时略有重叠。

同样的操作作用在 glm-4-flash 上更有诊断价值。来源标签与句式都未能撼动它的“取首条”规则（两者均为 0.0%），而把值在句中的位置改掉，误用率就升到 35.0%。可见值的位置是唯一同时作用于两个模型的表述杠杆，这也是我们把它当作首要表面因素、而非文字风格细节的原因。

表 3　单因素表述扰动下的过期误用率（%），无时间戳、更新在前、不复习基线。

表述变体	deepseek-chat	glm-4-flash
统一模板（基线）	100.0	0.0
仅来源标签变化	90.4	0.0
仅句式变化	85.6	0.0
仅值的位置变化	70.2	35.0
三者同时混杂	65.0	—

4.4　维护只是清除冲突，仅此而已

表 1 中的复习策略为什么会有一点效果？归档计数直接给出了机制。在无时间戳、更新在前的条件下，四种策略在同一预算内复习的记录数基本相同（0、41.3、41.0、41.5），但归档数差异很大：0、0、17.8、21.0 条，即 40 对冲突中的 0%、0%、44.5%、52.5%。对应的误用率为 100.0%、100.0%、55.4%、47.5%。

两者关系是精确的：误用率 = 1 − 归档覆盖率。冲突一旦被清除，上下文中只剩一条记录，答案必然正确；冲突只要还在，位置规则就起作用，答案必然是过期值。因此衰减信号本身没有做任

何额外的工作：完全忽略衰减的重要度优先，落在了同一条线上。维护只有在删除矛盾时才有帮助。

表 4　相同复习预算下的复习与归档结果，模板语料、无时间戳、deepseek-chat。覆盖率为归档冲突对数除以 40。

策略	复习数	归档数	覆盖率	误用率
不复习	0.0	0.0	0%	100.0
最近优先	41.3	0.0	0%	100.0
重要度优先	41.0	17.8	44.5%	55.4
衰减优先	41.5	21.0	52.5%	47.5

4.5　真实日志复现了同样的模式

表 2 末行把合成取值换成 13 条来自会话日志的真实更新链，记录按原始日志行呈现。模式依然成立：更新记录在前时误用率 89.1%，在后时 8.3%，带日期时 0.0%。比率比合成语料略低，可能是因为真实日志行的格式本身略有差异，但干预方式相同，条件之间的排序也相同。

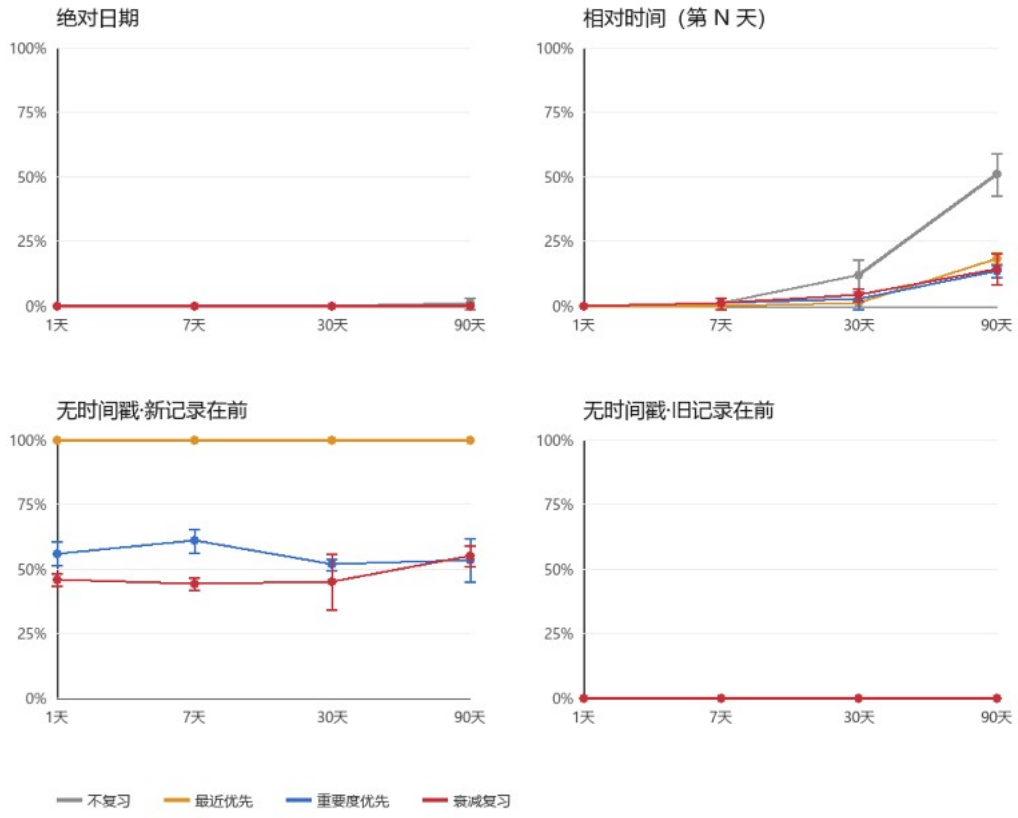


图 1　四种呈现条件下误用率随延迟的变化（deepseek-chat，40 条链 × 3 个种子），误差棒为跨种子的 95% 置信区间。

4.6 线索锚定在上下文末尾

位置结论引出一个机制性问题：模型敏感的是两条记录的先后，还是“最后一个值”离问题有多近？我们用插入中性句的方式把两者分开。这些句子不含任何数字（“巡检备注：机房温度正常，未发现异常情况。”）。在“间隔”条件下，填充句放在两条记录之间；在“尾部”条件下，放在两者之后。如果线索是两条记录之间的间距，尾部条件应当没有影响；如果线索是与问题的邻近程度，则间隔条件应当没有影响。

间隔条件完全无效：在我们测试的每个 k 值（1 与 8）上，误用率都保持 100%，因为过期值仍然是上下文中最后一条事实。尾部条件则立刻打破了规则：$k = 1$ 时误用率降到 83.1 ± 3.0，$k \le 8$ 时在 89.4 与 94.8 之间波动，$k = 16$ 时降到 62.3 ± 3.2（图 2）。二者并非平滑衰减：在末尾追加文本会先轻微削弱线索，越过某个阈值后则显著破坏它。

我们据此认为，线索锚定在上下文末尾：真正重要的是哪个值离问题最近，而这个锚点对追加在其后的任何文本都很脆弱。这同时排除了另一种解释——模型在比较两条记录、并按某种距离度量偏好较晚的一条：因为把两者隔开完全不产生影响。

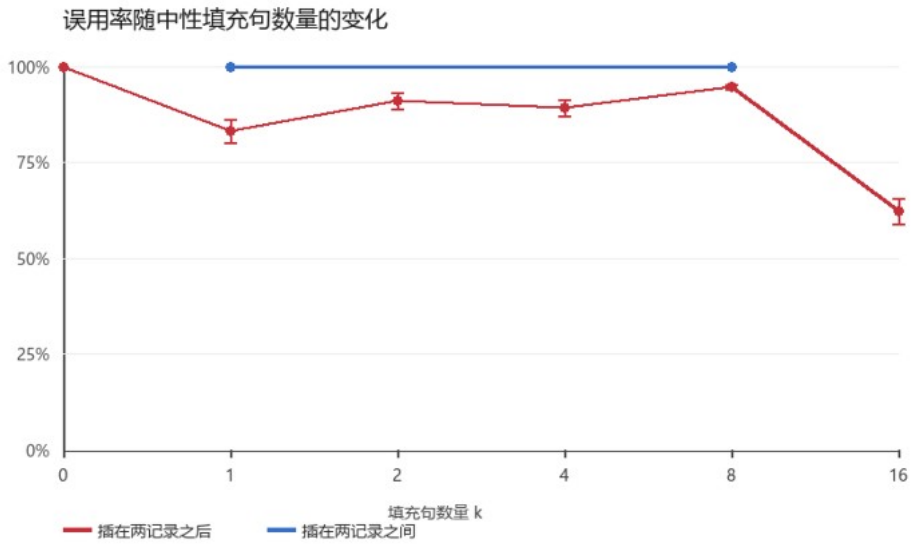


图 2　在两记录之后（红）或之间（蓝）加入中性填充句时的误用率；无时间戳、统一表述、deepseek-chat，40 条链 × 3 个种子，不复习。误差棒为跨种子与延迟的 95% 置信区间。

5　可判定性感知的记忆层

上述测量可以归纳为记忆系统的少量规则。我们把它表述为一个位于存储与提示词之间的层，并对原型做了端到端检验。

5.1 原则

P1，为一切记录打时间戳。每条记录都携带它“何时成立”的时间，并与记录一同呈现。没有时间的记录不可判定，一个日期比任何排序信号都更有价值。

P2，归档冲突，而不是复习它。新记录取代旧记录时，把旧记录标记为已归档，而不是让它继续存活、等待复习。4.4 节表明，这正是复习策略唯一起作用的机制；主动归档也比事后才发现冲突更便宜。

P3，把权威记录放在该模型的偏好端。当时间戳缺失或不可信时，模型会退回到位置规则，而方向因模型而异；因此这个放置位置应当按模型实测，而不是凭直觉假设。统一模板能让该规则保持确定——但只有当放置位置正确时，确定性才是好事。

5.2 接口

该层暴露三个操作。write(slot, value, t) 追加一条记录，并把同一槽位仍存活的记录标记为已被取代。render(budget, model) 按目标模型偏好的顺序、以统一模板与时间戳前缀输出存活记录，预算不足时按相关性截断。maintain(budget) 把维护预算花在归档上，而不是复习上，并返回清除的冲突数，使运维方能看到由此得到的覆盖率。

5.3 端到端检验

我们用约两百行代码实现了原型，并设计了一个多轮配置任务：五个服务、每个两个属性，跨八轮共十次更新，最后要求给出当前的部署清单。四种设计共享同一记录集合，仅在呈现与冲突处理上不同；每种设计在 40 个任务实例（20 个场景 × 2 个种子）上评测，指标为清单完全正确、字段正确率、以及携带已取代取值的字段比例。

表 5　端到端多轮配置任务：清单完全正确率、字段正确率与过期字段比例，每种设计 40 个实例。

设计	完全正确	字段正确率	过期字段
追加式，最新在前	0.0%	44.5%	55.5%
追加式，最旧在前	100.0%	100.0%	0.0%
时间戳，保留全部记录	100.0%	100.0%	0.0%
DAML（归档冲突）	100.0%	100.0%	0.0%

朴素设计由顺序而非内容决定：同一批记录，仅因最新记录所处的一端不同，完全正确率就是 0% 或 100%。加上时间戳后，这种依赖完全消失。DAML 在准确率上与时间戳方案持平，但只输出 18 条记录中的 10 条，记忆 token 减少 44%，因为已取代的记录被归档而不是继续携带。

该试验规模很小、任务也是合成的；我们把它当作“这些规则在端到端情形下依然成立”的存在性证明，而不是基准。面向多跳任务、更多模型与真实部署的完整评测，留待后续工作。

6 讨论

可判定性是设计属性，而不是模型能力。就某种呈现而言，一条记忆记录是可判定的，当且仅当上下文含有足够信息，能够判定两条相互矛盾的记录中哪一条是当前的。日期让存储变得可判定。位置不能：它把一个认识论问题变成了表层启发式，而这个启发式在不同模型上指向不同方向。正是表述的统一性让该启发式具有确定性，这也是同一份存储在某个模型上 100% 误用、在另一个模型上 0% 误用的原因。

其工程含义是具体的。为每条记录保存时间戳并渲染出来；没有日期的事实，不是智能体能可靠使用的事实。按实测而非直觉来选择最新记录的放置端，因为“最新在后”对我们测试的 DeepSeek 模型是对的，对 GLM-4-Flash 则恰好相反。如果依赖顺序，就应保持记录模板同质，并记住值在句中的位置本身也是一个杠杆。最后，把维护预算用于解决冲突而不是复习：一条从不归档已取代记录的复习策略，无论排序信号多么有原则，都不会提升准确率。

这些结果还提示了一种评测做法。智能体记忆的评测通常固定呈现方式、只改变策略；而我们的数据显示，呈现方式足以覆盖结果的全部取值范围。因此注入矛盾的基准至少应报告三件事：是否存在时间戳、冲突记录如何排序、表述是否统一。缺了这三个数字，记忆系统之间的比较就无法解释。

还有一个结果值得强调：关于衰减的否定结论。衰减优先在我们的网格里表现不错，但只是因为它恰好挑中了旧记录，而旧记录正是承载冲突的那些。任何具有同样偏好的策略都会得到相同结果。因此，“遗忘曲线能改善智能体记忆”[2]这类主张，应当先与“清除冲突”这一基线对照，再把功劳记到曲线头上。

7 局限

我们的任务刻意最小化：单跳事实、只有一条矛盾记录、中文提示、简短回答。多跳问题、更长的冲突链、以及需要聚合多条记录的任务可能表现不同，我们对此不作任何断言。真实日志语料规模很小（13 条链），且只覆盖单一属性（本地端口）。四个模型配置来自两家厂商，不足以说明各种位置规则的普遍程度；“方向因模型而异”这一发现在样本内是稳健的，但模型总体并非如此。复习被实现为确定性的调和规则，这隔离了选择问题，却抽象掉了 LLM 实际改写记忆时的成本与失效模式。此外，GLM-4-Plus 接近 50% 的行为仍未解释，我们如实报告而不作解读。

8 结论

当智能体的记忆中含有相互矛盾的记录时，它给出的答案，更多取决于这些记录能否被裁决，而不是记忆被如何维护。日期能裁决，顺序也能裁决，但方向取决于模型；表述的统一性则决定顺序规则是否被确定地执行。维护策略的作用仅限于清除冲突，一旦控制住这一点，其表面收益便不复存在。记忆系统应当把可判定性作为一等属性来构建，也以此为准来评测。

参考文献

PARK J S, O'BRIEN J C, CAI C J, 等. Generative agents: interactive simulacra of human behavior[C]//UIST. 2023. https://arxiv.org/abs/2304.03442.

ZHONG W, GUO L, GAO Q, 等. MemoryBank: enhancing large language models with long-term memory[C]//AAAI. 2024. https://arxiv.org/abs/2305.10250.

PACKER C, WOODERS S, LIN K, 等. MemGPT: towards LLMs as operating systems[R]. 2023. https://arxiv.org/abs/2310.08560.

LIU N F, LIN K, HEWITT J, 等. Lost in the middle: how language models use long contexts[J]. TACL, 2024. https://arxiv.org/abs/2307.03172.

MAHARANA A, LEE D H, TULYAKOV S, 等. Evaluating very long-term conversational memory of LLM agents[R]. 2024. https://arxiv.org/abs/2402.17753.

WU D, WANG H, YU W, 等. LongMemEval: benchmarking chat assistants on long-term interactive memory[R]. 2024. https://arxiv.org/abs/2410.10813.

SETTLES B, MEEDER B. A trainable spaced repetition model for language learners[C]//ACL. 2016. https://aclanthology.org/P16-1177/.

LEWIS P, PEREZ E, PIKTUS A, 等. Retrieval-augmented generation for knowledge-intensive NLP tasks[C]//NeurIPS. 2020. https://arxiv.org/abs/2005.11401.

SHINN N, CASSANO F, GOPINATH A, 等. Reflexion: language agents with verbal reinforcement learning[C]//NeurIPS. 2023. https://arxiv.org/abs/2303.11366.

WANG G, XIE Y, JIANG Y, 等. Voyager: an open-ended embodied agent with large language models[R]. 2023. https://arxiv.org/abs/2305.16291.

BORGEAUD S, MENSCH A, HOFFMANN J, 等. Improving language models by retrieving from trillions of tokens[C]//ICML. 2022. https://arxiv.org/abs/2112.04426.

EBBINGHAUS H. Über das Gedächtnis[M]. 1885. https://archive.org/details/berdasgedchtni00ebbi.